

constraints observed during benchmark analysis (Section 2.2). Empirical testing using high-precision timers (`System.nanoTime()`) shows that runtimes fluctuate early on due to JVM caching and background garbage collection (Section 2.2 & 2.6).
Knowing how data behaves allows me to choose the perfect data structure.

[Reply](#) | 



Minh Chi Hoang (<https://sjeccd.instructure.com/courses/53429/users/226615>)

Sep 12 3:18am | Last reply Sep 12 3:09pm



When choosing a data structure for a program I would first think about what the program needs to do with the data. I would look at the type of data I have, the operations I need to perform, and how often those operations will be used. Algorithm Analysis can help me compare different choices and understand which one will make my program more efficient. First, I would consider is time complexity. I would look at how long common operations such as searching, inserting, deleting, and accessing data take as the amount of data increases. Like if I need to access an item by its position very often, an array would be a good choice because accessing an element by its index is usually $O(1)$. This means I can access an element directly without searching through the other elements. Then if my program frequently adds or removes items from a list, I would consider whether another data structure would be more suitable. For example, a linked list can make insertion and deletion easier in certain situations. However, accessing an element by its position in a linked list can take $O(n)$ time because the program may need to go through the elements one by one. This shows why I need to consider what operations my program performs most frequently instead of simply choosing the fastest data structure overall. I would also consider the amount of data my program needs to handle. A data structure that works well for 20 items may not be the best choice for millions of items. As the amount of data becomes larger, the difference between $O(1)$, $O(\log n)$, and $O(n)$ can become much more noticeable. I would also consider memory usage because some data structures require additional memory to store information or connections between elements. Finally, I would choose a data structure by considering the program's main operations, time complexity, amount of data, memory usage, and how frequently the data changes. The best choice is not always the most complicated one. I would choose the structure that meets the program's needs while keeping the code efficient and understandable.

[Reply](#) | 

> [1 Reply \(1\)](#)



Vinh Mach (*Use my name as pronoun*)